D3Blocks:

Biblioteka języka Python do tworzenia interaktywnych i samodzielnych wykresów D3js.
Twórz interaktywne, autonomiczne i atrakcyjne wizualnie wykresy oparte na grafice d3 javascript (d3js), ale konfigurowalne za pomocą języka Python.

Python stał się jednym z najpopularniejszych języków programowania do analizy i wizualizacji danych. Wizualizacja może być kluczem do sukcesu w projektach, ponieważ może ujawnić ukryte spostrzeżenia w danych i poprawić zrozumienie. **Najlepszym sposobem na zrozumienie i wyjaśnienie danych jest uczynienie ich interaktywnymi**. Mimo że wiele pakietów wizualizacji jest dostępnych w języku Python, nadal trudno jest tworzyć piękne, autonomiczne i interaktywne wykresy, które mogą również działać poza własnym komputerem.

Dzięki D3Blocks wykraczamy poza pojedynczy wykres i tworzymy framework, który zawiera teraz 10 pięknych wykresów. *Jest open-source i nie ma potrzeby instalowania niczego innego niż Python do tworzenia wykresów D3. Wynikiem jest interaktywny wykres, do którego potrzebna jest tylko przeglądarka (internetowa); Udostępnianie i publikowanie staje się zatem bardzo łatwe*. ***Przedstawię D3Blocks i krótko opiszę 10 bloków z praktycznymi przykładami.***

Nazwa "D3" jest skrótem od **Data-Driven Documents**, która jest **biblioteką JavaScript** do tworzenia dynamicznych, interaktywnych wizualizacji danych, jest szybka, piękna do zobaczenia i możesz dodać całą swoją kreatywność. Co więcej, nie potrzebujesz żadnej innej technologii niż przeglądarka do wykreślania wykresów.

*"**D3 pomaga ożywić dane za pomocą HTML, SVG i CSS**. Nacisk D3 na standardy internetowe daje pełne możliwości nowoczesnych przeglądarek bez wiązania się z zastrzeżoną strukturą, łącząc potężne komponenty wizualizacji i oparte na danych podejście do manipulacji DOM. " - d3js.org*

D3 jest również nazywany D3.js lub d3js.

Każdy wykres utworzony przez D3Blocks jest całkowicie zamknięty w jednym pliku HTML, który umożliwia udostępnianie i publikowanie, do którego nie potrzebujesz żadnej innej technologii niż przeglądarka.

ZADANIE

Zainstaluj D3Blocks i wykonaj 10 wizualizacji. Prześlij do TEAMS inną wersję wykresu niż wyjściowa (tam, gdzie będzie to możliwe).

```
pip install d3blocks
```

Network graph.

Za pomocą bloku D3Graph możemy wykreślić interakcje sieciowe za pomocą wykresów D3 skierowanych na siłę W sekcji kodu 1 ładujemy zbiór danych energetycznych [1], a następnie analizujemy dane za pomocą bloku d3graph. Węzły są pokolorowane heurystyką grupowania Louvain, a szerokość krawędzi zależy od wag. Sieć można teraz eksplorować w sposób interaktywny, w którym krawędzie można łamać na podstawie wytrzymałości krawędzi (za pomocą suwaka umieszczonego u góry). Dzięki temu najsilniejsze połączone węzły pozostają nienaruszone.

```
from d3blocks import D3Blocks

# Initialize
d3 = D3Blocks()

# Import example
df = d3.import_example('energy')

# Show the input data
print(df)
#                     source           target   weight
# 0      Agricultural 'waste'    Bio-conversion  124.729
# 1            Bio-conversion           Liquid    0.597
# 2            Bio-conversion           Losses   26.862
# 3            Bio-conversion            Solid  280.322
# 4            Bio-conversion              Gas   81.144
# ..                      ...              ...      ...
# 63       Thermal generation  District heating   79.329
# 64                    Tidal  Electricity grid    9.452
# 65  UK land based bioenergy    Bio-conversion  182.010
# 66                     Wave  Electricity grid   19.013
# 67                     Wind  Electricity grid  289.366

# [68 rows x 3 columns]


# Initialize Network chart but do not yet show the chart.
d3.d3graph(df, showfig=False)

# Color node on clustering
d3.D3graph.set_node_properties(color='cluster')

# Make adjustments to the node: Thermal_generation
d3.D3graph.node_properties['Thermal_generation']['size']=20
d3.D3graph.node_properties['Thermal_generation']['edge_color']='#000fff' # Blue node edge
d3.D3graph.node_properties['Thermal_generation']['edge_size']=3 # Node-edge Size

# Make adjustments to the edge: 'Solar', 'Solar_Thermal'
d3.D3graph.edge_properties['Solar', 'Solar_Thermal']['color']='#000fff'
d3.D3graph.edge_properties['Solar', 'Solar_Thermal']['weight_scaled']=10

# Show the network graph
d3.D3graph.show()
```

Wykres Sankeya

Wykres Sankeya to wizualizacja przedstawiająca przepływ z jednego zestawu wartości do drugiego. Prostokąty reprezentują węzły, a szerokość strzałek jest proporcjonalna do natężenia przepływu. Sankeye są najlepiej używane, gdy chcesz pokazać relacje wiele-do-wielu lub odkryć wiele ścieżek za pomocą zestawu etapów. Na przykład, jak ruch przepływa ze stron (początkowych) do innych stron w Twojej witrynie. W celu demonstracji wykorzystuje się zbiór danych dotyczących energii .

```
from d3blocks import D3Blocks

# Initialize
d3 = D3Blocks()

# Import example
df = d3.import_example('energy')

# Link settings
d3.sankey(df, link={"color": "source-target"})
```

Chord graph.

Wykres akordowy przedstawia przepływy lub połączenia między kilkoma jednostkami lub węzłami. Każda jednostka jest reprezentowana przez fragment w zewnętrznej części okrągłego układu. Następnie między poszczególnymi jednostkami rysowane są łuki. Rozmiar łuku jest proporcjonalny do znaczenia przepływu. W celu demonstracji zestaw danych energetycznych jest używany jako wejście dla bloku akordów (sekcja kodu 3). Jako punkt wyjścia do rozwoju bloku Chord, kod d3js jest rozwidlony od Mike'a Bostocka, a następnie Pythonized.

```
from d3blocks import D3Blocks

# Initialize
d3 = D3Blocks()

# Import example
df = d3.import_example('energy')

# Link settings
d3.chord(df, filepath='chord_demo.html')
```

Wykresy do wykreślania danych szeregów czasowych.

W danych szeregów czasowych musi istnieć element DateTime wraz ze stanem w danym momencie. Najprostszym przykładem szeregów czasowych są dane giełdowe, które można wizualizować za pomocą bloku szeregów czasowych. Alternatywnie mogą również istnieć zależności czasowe, w których jedno działanie następuje po drugim (np. Jak proces). W takich przypadkach wykres Moving Bubbles jest bardziej wnikliwy, aby zrozumieć ruchy i to, czy klastry występują w określonych punktach czasowych.

Time Series.

The *TimeSeries* graph can be used in case a date-time element is available, and where the values directly follow up with each other. With the *TimeSeries* block, you can enable/disable columns of interest, and perform brushing and zooming to quickly focus on regions of interest. An example in *code section 5* is shown where various climate variables are visualized (*Figure 5)*. As a starting point to develop the *Timeseries* block, the d3js code is forked from Mike Bostock and then Pythonized.

```
# Import library.
from d3blocks import D3Blocks

# Initialize and set the datetime format
d3 = D3Blocks()

# Import climate dataset
df = d3.import_example('climate')

# Print
df.head()

#           date   meantemp   humidity  wind_speed  meanpressure
#     2017-01-01  15.913043  85.869565    2.743478     59.000000
#     2017-01-02  18.500000  77.222222    2.894444   1018.277778
#     2017-01-03  17.111111  81.888889    4.016667   1018.333333
#     2017-01-04  18.700000  70.050000    4.545000   1015.700000
#     2017-01-05  18.388889  74.944444    3.300000   1014.333333
#           ...        ...        ...         ...           ...
#     2017-04-20  34.500000  27.500000    5.562500    998.625000
#     2017-04-21  34.250000  39.375000    6.962500    999.875000
#     2017-04-22  32.900000  40.900000    8.890000   1001.600000
#     2017-04-23  32.875000  27.500000    9.962500   1002.125000
#     2017-04-24  32.000000  27.142857   12.157143   1004.142857

# [114 rows x 5 columns]

# Create the timeseries chart.
d3.timeseries(df, datetime='date', dt_format='%Y-%m-%d %H:%M:%S', fontsize=10)
```

Ruchome bąbelki.

Wykres MovingBubbles zapewnia wgląd w to, jak działania następują po sobie w czasie. Może pomóc w zrozumieniu ruchów jednostek oraz tego, czy klastry występują w określonych punktach czasowych i stanach. Chociaż metoda wizualizacji nie jest najbardziej wydajna, świetnie sprawdza się w przypadku węzłów skierowanych na siłę i zderzających się. Wykres statyczny jest rozwidlony z postu o nazwie "Dzień z życia Amerykanów", a następnie mocno zmieniony, aby upewnić się, że wykres może uogólniać się na różne typy zestawów danych, liczbę stanów, orientację, formatowanie, kolorowanie itp. Stworzyliśmy funkcję do generowania losowego zestawu danych z różnymi stanami i w losowych punktach czasowych.

```
# Import library
from d3blocks import D3Blocks

# Set color scheme
d3 = D3Blocks()

# Generate random data with various states
df = d3.import_example('random_time', n=10000, c=500, date_start="1-1-2000 00:10:05", date_stop="1-1-2001 23:59:59")

#                  datetime sample_id     state
# 0    2000-01-01 00:10:10        54  Hospital
# 1    2000-01-01 00:10:19        28      Home
# 2    2000-01-01 00:10:21        98  Hospital
# 3    2000-01-01 00:10:30        12  Hospital
# 4    2000-01-01 00:10:32        71    Travel
#                   ...       ...       ...
# 9995 2000-01-01 23:59:20         2  Hospital
# 9996 2000-01-01 23:59:24        17      Home
# 9997 2000-01-01 23:59:31        37     Bored
# 9998 2000-01-01 23:59:47        48     Sport
# 9999 2000-01-01 23:59:47        70    Travel

# [10000 rows x 3 columns]

# Make the moving bubbles chart.
d3.movingbubbles(df,
                 datetime='datetime',
                 state='state',
                 sample_id='sample_id',
                 standardize=None,
                 speed={"slow": 1000, "medium": 200, "fast": 10},
                 cmap='Set1',
                 filepath='movingbubbles.html')
```

ImageSlider.

ImageSlider jest świetny, gdy chcesz porównać dwa obrazy. Jest to najbardziej przydatne w przypadku stanu przed i po. Do celów demonstracyjnych użyłem zdjęcia Markusa Spiske z Unsplash. Dwa obrazy można teraz łatwo porównać (sekcja kodu 7, rysunek 7). Kod javascript jest rozwidlony z JohnEdChristensen, a następnie Pythonized, aby łatwo porównywać obrazy.

```
# Import
from d3blocks import D3Blocks

# Initialize
d3 = D3Blocks()

# Import example
img_before, img_after = d3.import_example('unsplash')

print(img_before)
'C:\\unsplash_before.jpg'

print(img_after)
'C:\\unsplash_after.jpg'

# Make image slider graph
d3.imageslider(img_before, img_after)
```

Wykres punktowy.

Wykres punktowy jest prawdopodobnie najbardziej znanym wykresem do wykreślania współrzędnych (x, y). Takie podstawowe wykresy są przydatne, szczególnie w przypadku możliwości szczotkowania i powiększania. Wykresy punktowe mogą być pokolorowane próbkami i używane do wykrywania relacji między próbkami (grupami). Danymi wejściowymi są współrzędne x i y, ale można ustawić różne inne parametry, takie jak etykieta klasy, rozmiar, kolory (krawędzi), informacje o podpowiedzi itp. Wykres punktowy może również tworzyć przejścia między dwoma lub trzema zestawami współrzędnych. Może to być pomocne podczas porównywania różnych zestawów mapowań, jak opisano w poprzednim poście. Do demonstracji wykorzystam zestaw danych dotyczących raka z projektu Cancer Genome Atlas (TCGA), który dostarcza wielu rodzajów danych molekularnych dla 34 różnych tkanek nowotworowych. Dane molekularne są przetwarzane i wyprowadzane są współrzędne t-SNE wraz z głównymi składnikami, które reprezentują podobieństwo między pacjentami. Dzięki wykresowi punktowemu możemy wizualizować oba zestawy współrzędnych (t-SNE i PC) i głębiej analizować wyniki.

```
# Import library
from d3blocks import D3Blocks

# Initialize
d3 = D3Blocks()

# import example
df = d3.import_example('cancer')

#          x          y   age  ... labels
#                              ...
#  37.204296  24.162813  58.0  ...    acc
#  37.093090  23.423557  44.0  ...    acc
#  36.806297  23.444910  23.0  ...    acc
#  38.067886  24.411770  30.0  ...    acc
#  36.791195  21.715324  29.0  ...    acc
#    ...         ...    ...   ...    ...
#   0.839383  -8.870781   NaN  ...   brca
#  -5.842904   2.877595   NaN  ...   brca
#  -9.392038   1.663352  71.0  ...   brca
#  -4.016389   6.260741   NaN  ...   brca
#   0.229801  -8.227086   NaN  ...   brca

# [4674 rows x 9 columns]

# Setup the tooltip
tooltip=df['labels'].values + ' <br /> Survival: ' + df['survival_months'].astype(str).str[0:4].values
# Set the size
size = df['survival_months'].fillna(1).values / 10

# Scatter
d3.scatter(df['x'].values,              # tSNE x-coordinates
           df['y'].values,              # tSNE y-coordinates
           x1=df['PC1'].values,         # PC1 x-coordinates
           y1=df['PC2'].values,         # PC2 y-coordinates
           scale=True,                  # Scale
           label_radio=['tSNE', 'PCA'], # Label radio button
           size=size,                   # Size
           color=df['labels'].values,   # Hex-colors or classlabels
           stroke='#000000',            # Edge color
           opacity=0.4,                 # Opacity
           tooltip=tooltip,             # Tooltip
           cmap='tab20',                # Colormap
           filepath='c://temp//scatter_demo.html')
```

Wykres skrzypiec.

Wykres skrzypiec wizualizuje rozkład zmiennych liczbowych dla jednej lub kilku grup. Jest to alternatywa dla wykresu pudełkowego, aby uzyskać wgląd w duże zbiory danych, w których wykres pudełka może ukryć część informacji. Za pomocą tego bloku możemy skonfigurować wykresy dla jednej lub kilku grup i zmienić różne elementy na wykresach, np. kolory, podpowiedzi i rozmiary pojemników.

```
# Import library
from d3blocks import D3Blocks

# Initialize
d3 = D3Blocks()

# import example
df = d3.import_example('cancer')

# Tooltip
tooltip=df['labels'].values + ' <br /> Survival: ' + df['survival_months'].astype(str).values

# Make the plot
d3.violin(x=df['labels'].values, # class labels on the x axis
          y=df['age'].values,    # Age
          tooltip=tooltip,       # Tooltip for hovering
          bins=50,               # Bins used for the histogram
          size=df['survival_months'].values/10, # Dotsize
          x_order=['acc','kich', 'brca','lgg','blca','coad','ov'], # Keep only these classes and plot in this order.
          figsize=[None, None],                                    # Figure size is automatically determined.
          filepath='violine_demo.html')
```

Wykres cząstek.

Dzięki blokowi cząstek możemy zamienić dowolne słowo w interaktywny wykres cząstek. Ruch myszy lub dotknięcie słowa pozwoli cząstce odbić się, a następnie powrócić do swoich pierwotnych miejsc. Zachowanie wykresu można zmieniać różne właściwości, takie jak odbicie, rozmiar cząstek i kolory. Przykład tworzenia wykresu cząstek pokazano w sekcji kodu 10 z wynikiem pokazanym na rysunku 10. Oryginalny javascript jest rozwidlony od Iana Johnsona, a następnie Pythonized.

```
# Import library
from d3blocks import D3Blocks

# Initialize
d3 = D3Blocks()

# Make particles
d3.particles('D3Blocks', collision=0.05, spacing=10, figsize=[1200, 500])
```